MYSQL

Table of Contents:-

-- 1. DATABASE, -- 2. TABLES
-- 3. INSERT ROWS
-- 4. SELECT
-- 5.UPDATE & DELETE
-- 7. AUTOCOMMIT, COMMIT, ROLLBACK
-- 8. CURRENT_DATE() & CURRENT_TIME()
-- 9. UNIQUE
-- 10. NOT NULL
-- 11. CHECK
-- 12. DEFAULT
-- 13. PRIMARY KEYS CONSTRAINTS
-- 14. AUTO-INCREMENT
-- 15. FOREIGN KEYS
-- 16. JOINTS
-- 17. FUNCTIONS
-- 18. AND, OR & NOT
-- 19. WILDCARD CHARACTERS [%]
-- 20. ORDER BY CLAUSE
-- 21. LIMIT BY CLAUSE
-- 22. UNION OPERATOR
-- 23. SELF JOIN
-- 24. VIEWS
-- 25. INDEX (BTree data structure)
-- 26. SUBQUERY
-- 27. GROUP BY CLAUSE
-- 28. ROLLUP CLAUSE
-- 29. ON DELETE
-- 30. STORED PROCEDURES
-- 31. TRIGGERS

Direction to use this pdf efficiently:-

1. Start with Basics
Begin from the Table of Contents and follow topics in order (DATABASE → TABLES → INSERT → SELECT). These form the foundation, so avoid skipping any section.

2. Learn Step-by-Step
Each topic builds on the previous one. After basics, move to constraints, keys, joins, and then advanced topics like views, indexes, and triggers.

3. Practice Alongside
Run every query shown and observe outputs. Learning SQL is practical, so hands-on practice is essential.

4. Use Navigation Links
Use the "Go to Table of Contents" links on each page to quickly jump between topics instead of scrolling.

5. Revise and Progress
Revisit earlier topics when needed and move forward systematically for better understanding.

From author:
I have learned this from BRO CODE Youtube.
Link:-

Author:Himanshu Yadav

-- 1. DATABASE 2. TABLES

```
create database myDB;
-- drop database myDB;
use myDB;

create table employees(
        employee_id int,
        first_name varchar(50),
    last_name varchar(50),
    hourly_pay decimal(5,2),
    hire_date date
);
select * from employees;

rename table employees to workers;
rename table workers to employees;

-- drop table employees;

alter table employees add phone_number int;
select * from employees;

alter table employees rename column phone_number to email_id;
select * from employees;

alter table employees modify column email_id varchar(100);
select * from employees;

alter table employees modify email_id varchar(100) after last_name;
select * from employees;

alter table employees modify email_id varchar(100) first;
select * from employees;

alter table employees drop column email_id;
select * from employees;
```

-- 3. INSERT ROWS

```
insert into employees
values(1, "Eugene", "Krabs", 25.50, "2023-01-02");
select * from employees;

insert into employees
values      (2, "Squidward", "Tentacles", 15.00, "2023-01-03"),
            (3, "Sponge", "Bob", 12.50, "2023-01-04"),
      (4, "Patrick", "Star", 12.50, "2023-01-05"),
      (5, "Sandy", "Cheeks", 17.25, "2023-01-06");
select * from employees;

-- ERROR in date formate
-- insert into employees
-- values    (0, "Squidward", "Tentacles", 15.00, "03-01-2023");
select * from employees;

insert into employees (employee_id, first_name, last_name)
values      (6, "Squidward", "Tentacles");
select * from employees;
```

-- 4. SELECT

```
select first_name, last_name from employees;

select * from employees
where employee_id = 3;

select * from employees
where first_name = "Sponge";

select * from employees
where hourly_pay >= 15;

select * from employees
where hire_date <= "2023-01-03";

select * from employees
```

```
where employee_id != 1;

select * from employees
where hire_date is null;
```

-- 5.UPDATE & DELETE

```
update employees
set first_name = "Sheldon",
        last_name = "Plankton"
where employee_id = 6;
select * from employees;

update employees
set hourly_pay = 10.00
where employee_id = 6;
select * from employees;

update employees
set hourly_pay = 10.5,
        hire_date = "2023-01-07"
where employee_id = 6;
select * from employees;

update employees
set hire_date = null
where employee_id = 6;
select * from employees;

delete from employees
where employee_id = 6;
select * from employees;
```

-- 7. AUTOCOMMIT, COMMIT, ROLLBACK

```
set autocommit = off;
delete from employees;
select * from employees;
```

```
rollback;
select * from employees;

set autocommit = on;
select * from employees;
```

-- 8. CURRENT_DATE() & CURRENT_TIME()

```
create table testTime(
        my_date date,
    my_time time,
    my_dateTime datetime
);
select * from testTime;

insert into testTime
values(current_date(), current_time(), now());
select * from testTime;
```

-- 9. UNIQUE

```
create table products(
        product_id int,
    product_name varchar(25),
    price decimal(4, 2)
);
alter table products
add constraint
unique(product_name);


insert into products
values        (100, "hamburger", 3.99),
              (101, "fries", 1.89),
       (102, "soda", 1.00),
       (103, "ice-cream", 1.49);
select * from products;
```

-- 10. NOT NULL

```
create table employees (
        product_id int,
    product_name varchar(25),
    price decimal(4, 2) not null
);
alter table products
modify column price decimal(4,2) not null;
select * from products;

insert into products
        values(104, "cookies", null);
select * from products;

delete from products
where product_id = 104;
select * from products;

alter table products
modify column price decimal(4,2) not null;
select * from products;

insert into products
        values(104, "cookies", null);
select * from products;

insert into products
        values(104, "cookies", 0);
select * from products;
```

-- 11. CHECK

```
select * from employees;
create table employees (
        employee_id int,
    first_name varchar(50),
    last_name varchar(50),
    hourly_pay decimal(5,2),
```

```
    hire_date date,
    constraint chk_hourly_pay check(hourly_pay >= 10.0)
);

 alter table employees
 add constraint chk_hourly_pay check(hourly_pay >= 10.0);

 insert into employees
 values (6, "Sheldon", "Plankton", 5.00, "2023-01-07"); -- Error Code:
3819. Check constraint 'chk_hourly_pay' is violated.

 delete from employees
 where employee_id = 6;
 select * from employees;

 alter table employees
 drop check chk_hourly_pay;

 insert into employees
 values(6, "Sheldon", "Plankton", 5.00, "2023-01-07");
 select * from employees;
```

-- 12. DEFAULT

```
insert into products
values        (104, "straw", 0.0),
              (105, "napkin", 0.0),
       (106, "fork", 0.0),
       (107, "spoon", 0.0);
select * from products;

delete from products
where product_name = "cookies";
select * from products;

delete from products
where product_id >= 104;
select * from products;
```

```
create table products(
        product_id int,
   product_name varchar(25),
   price decimal(4, 2) default 0.00
);

alter table products
alter price set default 0.0;
select * from products;

 insert into products (product_id, product_name)
 values       (104, "straw"),
              (105, "napkin"),
       (106, "fork"),
       (107, "spoon");
select * from products;

create table transactions(
        transaction_id int,
   amount decimal(5, 2),
   transction_date datetime default now()
);
select * from transactions;

insert into transactions (transaction_id, amount)
values       (1, 4.99);
insert into transactions (transaction_id, amount)
values       (2, 2.89);
insert into transactions (transaction_id, amount)
values (3, 8.37);
select * from transactions;
```

-- 13. PRIMARY KEYS CONSTRAINTS

```
create table transactions(
        transaction_id int primary key,
   amount decimal(5, 2)
);
alter table transactions
```

```
add constraint
primary key(transaction_id);
-- ONLY ONE PRIMARY KEY FOR EACH TABLE
select * from transactions;

insert into transactions (transaction_id, amount)
values       (1000, 4.99);
select * from transactions;

insert into transactions(transaction_id, amount)
values       (1001, 2.89);
select * from transactions;

insert into transactions(transaction_id, amount)
values (1002, 3.38);
select * from transactions;

insert into transactions(transaction_id, amount)
values (null, 4.99); -- CANNOT BE NULL and SHOULD BE UNIQUE
select * from transactions;

insert into transactions(transaction_id, amount)
values (1003, 4.99);
select * from transactions;

select amount from transactions
where transaction_id = 1003; -- TO DO REFUND FOR A SPECIFIC
CUSTOMER
```

-- 14. AUTO-INCREMENT

```
create table transactions2(
        transaction_id int primary key auto_increment,
   amount decimal(4, 2),
   transaction_date datetime default now()
);
select * from transactions2;

insert into transactions2 (amount)
```

```
values (4.99);
select * from transactions2;

insert into transactions2 (amount)
values (2.89);
select * from transactions2;

insert into transactions2 (amount)
values (3.38);
select * from transactions2;

insert into transactions2 (amount)
values (4.99);
select * from transactions2;

alter table transactions2
auto_increment = 1000;

delete from transactions2;
select * from transactions2;

insert into transactions2 (amount)
values (4.99);
select * from transactions2;

insert into transactions2 (amount)
values (2.89);
select * from transactions2;

insert into transactions2 (amount)
values (3.38);
select * from transactions2;

insert into transactions2 (amount)
values (4.99);
select * from transactions2;
```

-- 15. FOREIGN KEYS

```
create table customers (
        customer_id int primary key auto_increment,
    first_name varchar(50),
    last_name varchar(50)
);
select * from customers;

insert into customers (first_name, last_name)
values        ("Fred", "Fish"),
              ("Larry", "Lobster"),
        ("Bubble", "Base");
select * from customers;

create table transactions3 (
        transaction_id int primary key auto_increment,
    amount decimal(5, 2),
    customer_id int,
    foreign key(customer_id) references customers(customer_id)
);
select * from transactions;

-- to drop foreign key we need to use
alter table transactions3
drop foreign key transactions3_ibfk_1;

alter table transactions3
add constraint fk_customer_id
foreign key(customer_id) references customers(customer_id);

-- delete from transactions
-- select * from transactions;

alter table transactions3
auto_increment = 1000;

insert into transactions3 (amount, customer_id)
values        (4.99, 3),
```

```
                (2.89, 2),
        (3.38, 3),
        (4.99, 1);
select * from customers;
select * from transactions3;

-- delete from customers
-- where customer_id = 3; -- Error Code: 1451. Cannot delete or update
a parent row: a foreign key constraint fails (`mydb`.`transactions3`,
CONSTRAINT `fk_customer_id` FOREIGN KEY (`customer_id`)
REFERENCES `customers` (`customer_id`))

-----------------------------------------------------------------
--- for my sake
create table atuoinc (
        incr_id int,
    incr_name varchar(20)
);
select * from atuoinc;
alter table atuoinc
-- modify incr_id int primary key;
modify incr_id int auto_increment; -- Error Code: 1075. Incorrect table
definition; there can be only one auto column and it must be defined as a
key
insert into atuoinc (incr_name)
values        ("two");
select * from atuoinc;
rename table autoinc to auto_inc;
-----------------------------------------------------------------
```

-- 16. JOINTS

: used to combine two or more rows based on related column between them such as "foreign key"

```
insert into transactions3 (amount, customer_id)
values        (1.00, null);
select * from transactions3;
```

```
insert into customers (first_name, last_name)
value ("Poppy", "Puff");
select * from customers;

--- INNER JOIN

select  *
from transactions3 inner join customers
on transactions3.customer_id = customers.customer_id;

select  transaction_id, amount, first_name, last_name
from transactions3 inner join customers
on transactions3.customer_id = customers.customer_id;

--- LEFT JOIN
select *
from transactions3 left join customers
on transactions3.customer_id = customers.customer_id;

--- RIGHT JOIN
select *
from transactions3 right join customers
on transactions3.customer_id = customers.customer_id;
```

-- 17. FUNCTIONS

```
select count(amount)
from transactions3;

select count(amount) as count
from transactions3;

select count(amount) as "today's transactions"
from transactions3;

select max(amount) as maximum
from transactions3;

select min(amount) as minimum
```

```
from transactions3;

select avg(amount) as average
from transactions3;

select sum(amount) as sum
from transactions3;

select * from employees;

select concat(first_name, " ", last_name) as full_name
from employees;
```

-- 18. AND, OR & NOT

```
alter table employees
add column job varchar(25) after hourly_pay;
select * from employees;

update employees
set job = "manager"
where employee_id = 1;
select * from employees;

update employees
set job = "cashier"
where employee_id = 2;
select * from employees;

update employees
set job = "cook"
where employee_id = 3;
select * from employees;

update employees
set job = "cook"
where employee_id = 4;
select * from employees;
```

```
update employees
set job = "asst. manager"
where employee_id = 5;
select * from employees;

update employees
set job = "janitor"
where employee_id = 6;
select * from employees;

select * from employees
where hire_date < "2023-01-05" and job = "cook";

select * from employees
where job = "cook" or job = "cashier";

select * from employees
where job = "cook" and job = "cashier"; -- both conditions should be
correct

select * from employees
where not job = "manager";

select * from employees
where not job = "manager" and not job = "asst. manager";

select * from employees
where hire_date between "2023-01-04" and "2023-01-07";

select * from employees
where job in ("cook", "cashier", "janitor");
```

-- 19. WILDCARD CHARACTERS [% _]

```
-- USED TO SUBSTITUTE ONE OR MORE CHARACTERS IN A
STRING

select * from employees
where first_name like "s%";
```

```
select * from employees
where hire_date like "2023%";

select * from employees
where last_name like "%r";

select * from employees
where first_name like "sp%";

select * from employees
where job like "_ook";

select * from employees
where hire_date like "____-01-03";

select * from employees
where job like "_a%";
```

-- 20. ORDER BY CLAUSE

```
select * from employees
order by last_name;
-- to reverse the alphabetical order for ascending asc(asc is default)
select * from employees
order by last_name desc;

select * from employees
order by first_name desc;

select * from employees
order by hire_date desc;
-- doing order by more than one column
select * from transactions3
order by amount asc, transaction_id;
```

-- 21. LIMIT BY CLAUSE

```
-- LIMIT clause is used to limit the number of records.
-- Useful if you're working with a lot of data.
-- Can be used to display a large data on pages (pagination).
-- CAN DISPLAY LOTS OF DATA ON DIFFERENT PAGES.

-- <-[00][01][02][03][04][05]->

select * from customers
limit 3;

select * from customers
order by last_name limit 1;

select * from customers
order by last_name desc limit 1;

select * from customers
limit 1,1; -- first number is off-set/ limit one record after first

select * from customers;

-- if i need the next set of 10 customers we can set off set of 10.alter
select * from customers
limit 10, 10; -- displays number after off-set here 10.
```

-- 22. UNION OPERATOR

```
-- combines the results of two or more select statements
-----------------------------------
create table income(
        income_name varchar(25),
    amount decimal(7, 2)
);
select * from income;
insert into income
values      ("orders", 10000.00),
            ("merchandise", 5000.00),
```

```
        ("services", 1250.00);
select * from income;
create table expenses(
         expense_name varchar(25),
    amount decimal(7, 2)
);
insert into expenses
values        ("wages", -25000.00),
              ("tax", -500.00),
        ("repairs", -15000.00);
select * from expenses;
-----------------------------------
select * from income
union
select * from expenses;

select first_name, last_name from employees
union                  -- if "UNION ALL" will also keep any duplicate values
select first_name , last_name from customers;

insert into customers
values(5, "Sheldon", "Plankton");
select * from customers;

select first_name, last_name from employees
union all              -- only UNION doesn't allows duplicate/ -- to do so use
UNION ALL
select first_name, last_name from customers;

delete from customers
where customer_id = 5;
select * from customers;
```

-- 23. SELF JOIN

```
-- join another copy of a table to itself
-- used to compare rows of the same table
-- helps to display a heirarchy of data
```

```
alter table customers
add referral_id int;
select * from customers;

update customers
set referral_id = 1
where customer_id = 2;
select * from customers;

update customers
set referral_id = 2
where customer_id = 3;
select * from customers;

update customers
set referral_id = 2
where customer_id = 4;
select * from customers;

select *
from customers as a
inner join customers as b
on a.referral_id = b.customer_id;

select a.customer_id, a.first_name, a.last_name, b.first_name,
b.last_name
from customers as a
inner join customers as b
on a.referral_id = b.customer_id;

select a.customer_id, a.first_name, a.last_name,
concat(b.first_name, " ",b.last_name) as "referred_by"
from customers as a
inner join customers as b
on a.referral_id = b.customer_id;

select a.customer_id, a.first_name, a.last_name,
concat(b.first_name, " ",b.last_name) as "referred_by"
from customers as a
```

```
left join customers as b
on a.referral_id = b.customer_id;

alter table employees
add supervisor_id int;
select * from employees;

update employees
set supervisor_id = 5
where employee_id = 2;
select * from employees;

update employees
set supervisor_id = 5
where employee_id = 3;
select * from employees;

update employees
set supervisor_id = 5
where employee_id = 4;
select * from employees;

update employees
set supervisor_id = 5
where employee_id = 6;
select * from employees;

update employees
set supervisor_id = 1
where employee_id = 5;
select * from employees;

select *
from employees as a
inner join employees as b
on a.supervisor_id = b.employee_id;

select a.first_name, a.last_name,
concat(b.first_name, " ", b.last_name) as "reports to"
```

```
from employees as a
inner join employees as b
on a.supervisor_id= b.employee_id;

select a.first_name, a.last_name,
concat(b.first_name, " ", b.last_name) as "reports to"
from employees as a
left join employees as b
on a.supervisor_id= b.employee_id;
```

-- 24. VIEWS

```
-- a virtual table based on the result-set of an SQL statement
-- The fields in a view are fields from one or more real tables in the
database
-- They're not real tables, but can be interacted with as if they were

select * from employees;

create view employee_attendence as
select first_name, last_name
from employees;

select * from employee_attendence;

drop view employee_attendence;

select * from customers;

alter table customers
add column email varchar(50);
select * from customers;

update customers
set email = "FFish@gmail.com"
where customer_id = 1;
select * from customers;

update customers
```

```
set email = "LLobster@gmail.com"
where customer_id = 2;
select * from customers;
-------------------------
update customers
set last_name = "bass"
where customer_id = 3;
-------------------------
update customers
set email = "BBass@gmail.com"
where  customer_id = 3;
select * from customers;

update customers
set email = "PPuff@gmail.com"
where customer_id = 4;
select * from customers;

create view customer_emails as
select email
from customers;

select * from customer_emails;

insert into customers
values       (5, "Pearl", "Krabs", null, "PKrabs@gmail.com");
select * from customers;

select * from customer_emails; -- it has been updated automatically
because we are using views from table
```

-- 25. INDEX (BTree data structure)

```
-- Indexes are used to find values within a specific column more quickly
-- MySQL normally searches sequentially through a column
-- The longer the column, the more expensive the operation is
-- UPDATE takes more time, SELECT takes less time

show indexes from customers;
```

```
create index last_name_idx
on customers(last_name);

show indexes from customers;

select * from customers
where last_name = "Puff"; -- this would be fast

select * from customers
where first_name = "Poppy"; -- slower in comparision to if index created
on.

-- creating multiple indexes

create index last_name_first_name_idx
on customers(last_name, first_name);

show indexes from customers;

alter table customers
drop index last_name_idx;
show indexes from customers;

select * from customers
where last_name = "Puff" and first_name = "Poppy";

select * from customers
where        first_name = "Poppy";
```

-- 26. SUBQUERY

```
-- a query within another query
-- query(subquery)

select * from employees;

update employees
set hourly_pay = 10.00
```

```
where employee_id = 6;

select * from employees;

select avg(hourly_pay) from employees;
-- using the above query as subquery

select first_name, last_name,  hourly_pay,(select avg(hourly_pay) from
employees) as avg_pay
from employees;

select first_name, last_name, hourly_pay from employees
where hourly_pay > (select avg(hourly_pay) from employees);

select * from transactions3;

select distinct customer_id from transactions3
where customer_id is not null;
-----------------------------------------------
select first_name, last_name from customers
where customer_id in
(select distinct customer_id from transactions3
where customer_id is not null);
-- same as above but in opposite.
select first_name,last_name from customers
where customer_id not in (1,2,3);
-----------------------------------------------
```

-- 27. GROUP BY CLAUSE

```
-- GROUP BY = aggregate all rows by a specific column
-- often used with aggregate functions
-- ex. SUM (), MAX (), MIN(), AVG(), COUNT ()

select * from transactions3;
alter table transactions3
add order_date date after customer_id;
select * from transactions3;
```

```
update transactions3
set order_date = "2023-01-01"
where transaction_id = 1008;
select * from transactions3;

update transactions3
set order_date = "2023-01-01"
where transaction_id = 1009;
select * from transactions3;

update transactions3
set order_date = "2023-01-02"
where transaction_id = 1010;
select * from transactions3;

update transactions3
set order_date = "2023-01-03"
where transaction_id = 1012;
select * from transactions3;

insert into transactions3 (amount, customer_id, order_date)
value        (2.49, 4, "2023-01-03"),
             (5.48, null, "2023-01-03");
select * from transactions3;

select sum(amount), order_date
from transactions3
group by order_date;

select max(amount), order_date
from transactions3
group by order_date;

select min(amount), order_date
from transactions3
group by order_date;

select avg(amount), order_date
from transactions3
```

```
group by order_date;

select count(amount), order_date
from transactions3
group by order_date;

select sum(amount), customer_id
from transactions3
group by customer_id;

select max(amount), customer_id
from transactions3
group by customer_id;

select min(amount), customer_id
from transactions3
group by customer_id;

select avg(amount), customer_id
from transactions3
group by customer_id;

select count(amount), customer_id
from transactions3
group by customer_id;

select count(amount), customer_id
from transactions3
group by customer_id
where count(amount) > 1; -- use instead having | vv |

select count(amount), customer_id
from transactions3
group by customer_id
having count(amount) > 1;

select count(amount), customer_id
from transactions3
group by customer_id
```

```
having count(amount) > 1 and customer_id is not null;
```

-- 28. ROLLUP CLAUSE

```
-- ROLLUP, extension of the GROUP BY clause
-- produces another row and shows the GRAND TOTAL (super-aggregate value)

select sum(amount), order_date
from transactions3
group by order_date with rollup;

select count(transaction_id), order_date
from transactions3
group by order_date with rollup;

select count(transaction_id) as "# of order", customer_id
from transactions3
group by customer_id with rollup;

select sum(hourly_pay), employee_id
from employees
group by employee_id with rollup;
```

-- 29. ON DELETE

```
-- ON DELETE SET NULL = When a FK is deleted, replace FK with NULL
-- ON DELETE CASCADE = When a FK is deleted, delete row

delete from customers
where customer_id = 4; -- Error Cannot delete or update a parent row: a foreign key constraint fails (`mydb`.`transactions3`, CONSTRAINT `fk_customer_id` FOREIGN KEY (`customer_id`) REFERENCES `customers` (`customer_id`))
select * from customers;

set foreign_key_checks = 0; -- now you can delete
delete from customers
```

```
where customer_id = 4; -- in refereced table that value is still existing
select * from customers; -- referring a customer that doesn't exist

-- using options [either we can set that value null or delete that complete
row].

-- while creating table we can set
CREATE TABLE transactions (
        transaction_id INT PRIMARY KEY,
    amount DECIMAL (5, 2),
    customer_id INT,
    order_date DATE,
        FOREIGN KEY (customer_id) REFERENCES customers
(customer_id)
        ON DELETE SET NULL
);

-- or we already have a table we can update existing table

-- first we need to drop that key by
alter table transactions3 drop foreign key fk_customer_id;

-- adding foreign key constraint to our transaction3 table
alter table transactions3
add constraint fk_customer_id
foreign key(customer_id) references customers(customer_id)
on delete set null;

delete from customers
where customer_id = 4; -- now it will set to null in reffered table if "foreign
key" is deleted from referencing table
select * from transactions3;
select * from customers;

-- lets get poppy puff in our customers table
insert into customers
values (4, "Poppy", "Puff", 2, "PPuff@gmail.com");
select * from customers;
```

```
-- now to use ON DELETE CASCADE
-- first we need to drop that foreign key from transactions3 table by
alter table transactions3 drop foreign key fk_customer_id;

-- adding foreign key constraint (ON DELETE CASCADE) to our
transaction3 table
alter table transactions3
add constraint fk_customer_id
foreign key(customer_id) references customers(customer_id)
ON DELETE CASCADE;
select * from transactions3;

update transactions3
set customer_id = 4
where transaction_id = 1013;

delete from customers
where customer_id = 4;
select * from transactions3; -- in table 1013 is gone
```

-- 30. STORED PROCEDURES

```
-- Stored procedure = is prepared SQL code that you can save
--                                    great if there's a query that you write often
-- reduces network traffic
-- increases performance
-- secure, admin can grant permission to use
-- increases memory usage of every connection

use myDB;
select distinct first_name, last_name
from transactions3
inner join customers
on transactions3.customer_id = customers.customer_id;

delimiter $$
create procedure get_customers()
begin
select * from customers;
```

```
end $$
delimiter ;

call get_customers;

drop procedure get_customers;

delimiter $$
create procedure find_customer(in id int)
begin
select * from customers
where customer_id = id;
end $$
delimiter ;

call find_customer(5);

drop procedure find_customer;

delimiter $$
create procedure find_customer(in f_name varchar(50), in l_name
varchar(50))
begin
select * from customers
where first_name = f_name and last_name = l_name;
end $$
delimiter ;

drop procedure find_customer;

select * from customers;
call find_customer("Larry", "Lobster");
```

-- 31. TRIGGERS

```
-- Trigger = When an event happens, do something
--          ex. (INSERT, UPDATE, DELETE)
--                    benefits =  checks data, handles errors, auditing tables
```

```
alter table employees
add column salary decimal(10, 2) after hourly_pay;
select * from employees;

update employees
set salary = hourly_pay * 2080;
select * from employees;

create trigger before_hourly_pay_update
before update on employees
for each row
set new.salary = (new.hourly_pay * 2080);

show triggers;

update employees
set hourly_pay = 50
where employee_id = 1;
select * from employees;

update employees
set hourly_pay = hourly_pay + 1;
select * from employees;

create trigger before_hourly_pay_insert
BEFORE INSERT ON employees
FOR EACH ROW
SET NEW.salary = (NEW.hourly_pay * 2080);
-- salary will also be automatically calculated
INSERT INTO employees
VALUES (6, "Sheldon", "Plankton", 10, NULL, "janitor", "2023-01-07",
"5");
SELECT * FROM employees;

DROP TABLE expenses;
CREATE TABLE expenses(
        expense_id INT PRIMARY KEY,
    expense_name VARCHAR(50),
    expense_total DECIMAL(10,2)
```

```
);
select * from expenses;

INSERT INTO expenses
VALUES    (1, "salaries", 0),
          (2, "supplies", 0),
      (3, "taxes", 0);
SELECT * FROM expenses;

UPDATE expenses
SET expense_total = (SELECT SUM(salary) FROM employees)
WHERE expense_name = "salaries";
SELECT * FROM expenses;

CREATE TRIGGER after_salary_delete
AFTER DELETE ON employees
FOR EACH ROW
UPDATE expenses
SET expense_total = expense_total - OLD.salary
WHERE expense_name = "salaries";

SELECT * FROM employees;

DELETE FROM employees
WHERE employee_id = 6;
SELECT * FROM expenses;

CREATE TRIGGER after_salary_insert
AFTER INSERT ON employees
FOR EACH ROW
UPDATE expenses
SET expense_total = expense_total + NEW.salary
WHERE expense_name = "salaries";

SELECT * FROM expenses;

INSERT INTO employees
VALUES    (6, "Sheldon", "Plankton", 10, NULL, "janitor", "2023-01-07",
5);
```

```
SELECT * FROM expenses;


drop trigger after_salary_update;


CREATE TRIGGER after_salary_update
AFTER UPDATE ON employees
FOR EACH ROW
UPDATE expenses
SET expense_total = expense_total + (NEW.salary - OLD.salary)
WHERE expense_name = "salaries"; -- be care full in names.

show triggers;

UPDATE employees
SET hourly_pay = 100
WHERE employee_id = 1;
SELECT * FROM expenses;

SELECT * FROM employees;
```

End